

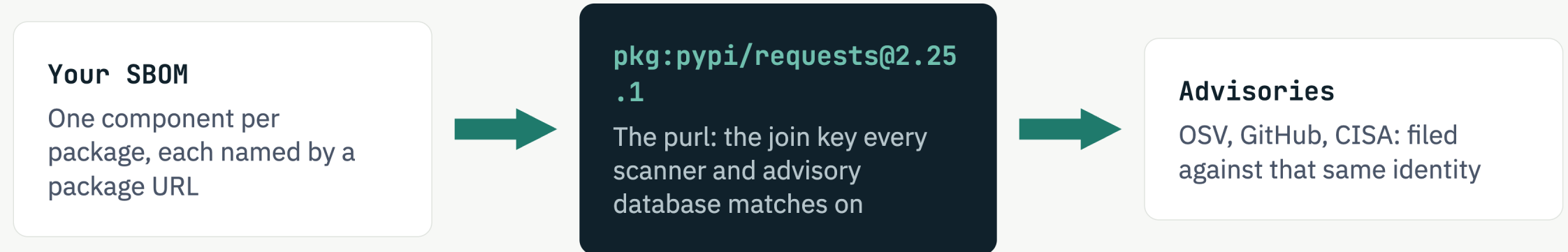
CONDA · PYPI · SBOMS

The hole in your SBOM

Why a bill of materials for a conda or pixi environment can be valid, complete-looking and nearly unsearchable, and what it takes to fix.

An SBOM is an inventory, joined to everything else by one key

Every package in a build: its name, version, supplier, hashes, license, and what depends on it. What it is for is a query: when an advisory lands, are we exposed?



The hole: the right package, under a name nobody searches

A pixi or conda environment is conda packages and PyPI packages, resolved together. NumPy comes from conda-forge, so a tool that does emit it names it the conda way.

What most tools write

`pkg:conda/numpy@2.3.1`

Correct, valid against the schema, and matched by almost no advisory database.

Where NumPy's advisories are filed

`pkg:pypi/numpy@2.3.1`

Same code, same version. The scanner only finds it under this name.

The document is present, well formed and hollow. Nobody chases an empty result that looks like a clean one.

How big the hole is

118 public conda and pixi workspaces, 15,722 packages, each looked up twice with only the identity changed.

+37.1%

1,274 advisories visible only
under the PyPI identity

45 of 96

affected workspaces showed
no advisories at all the
conventional way

0

advisories lost by adding the
PyPI identity

The fix: give every package both of its names

pixi-sbom reads the lockfile and writes CycloneDX or SPDX. A conda package that is a Python project keeps its conda identity and also gets its PyPI one.

```
numpy 2.3.4, from conda-forge
```

```
pkg:conda/numpy@2.3.4?build=py312h33ff503_0&channel=conda-forge
```

```
pkg:pypi/numpy@2.3.4
```

The PyPI name comes from the mapping conda-forge publishes, matched by the package's hash. A scanner can match either one.

One command, one document

```
$ pixi sbom --lockfile examples/projects/pixi/01-django/pixi.lock -p linux-64 --  
report packages
```

Summary: 66 packages, 9 declared by the workspace

```
$ pixi sbom --lockfile examples/projects/pixi/01-django/pixi.lock -p linux-64 --  
output sbom.cdx.json
```

A pixi workspace with Django on an old LTS: conda packages, PyPI packages, an editable local package. The interpreter and its system libraries are in the inventory too.

Find what is known, rank it, fail the build

```
$ pixi sbom --lockfile examples/projects/uv/02-flask/uv.lock -p linux-64 --vulnerabilities osv  
--kev --epss --report vulnerabilities
```

Summary: 41 findings in 8 packages

```
$ pixi sbom --lockfile examples/projects/uv/02-flask/uv.lock -p linux-64 --vulnerabilities osv  
--fail-on-severity high --output sbom.cdx.json
```

Gate failed: vulnerabilities (17). Exiting 4.

CISA's KEV marks what is exploited now; FIRST's EPSS scores how likely the rest is. Each gate has its own exit code, and the document is written either way.

Check a supplier's SBOM before you accept it

```
$ pixi sbom --from-sbom vendor.cdx.json --report quality
Quality: 77 of 100 for 24 packages
$ pixi sbom --from-sbom vendor.cdx.json --min-quality 80 --output -
Gate failed: quality (1). Exiting 10.
$ pixi sbom --lockfile examples/projects/uv/02-flask/uv.lock -p linux-64 --vulnerabilities osv
--vex-in examples/lab/vendor.openvex.json --report vulnerabilities
Summary: 40 open findings in 8 packages, 1 ignored
```

Score what you were sent, gate on it, and take a supplier's "not affected" statements as VEX, with their reasons kept in the record.

Not only pixi: any locked Python project

Your project has	Point it at
pixi or conda-lock	pixi.lock, conda-lock.yml
uv	uv.lock
Poetry, PDM or PEP 751	poetry.lock, pdm.lock, pylock.toml
pip, pinned	requirements.txt
Someone else's SBOM	--from-sbom file.cdx.json

Without pixi too: `pip install pixi-sbom`, `uvx pixi-sbom`, or a signed binary from the release.

In CI: one step, a gate, an attached document

```
- uses: millsk/pixi-sbom@v1
with:
  vulnerabilities: osv
  kev: "true"
  fail-on-kev: "true"
  fail-on-severity: high
```

4

a vulnerability past the line

3

a license the policy forbids

6

a change from the last SBOM

10

a document too thin to trust

Each gate has its own exit code, so the pipeline can tell what failed. The document is written either way.

What it does not do

Prove you are safe

It finds what is already published. An advisory that does not exist yet is not in any database.

See what is not locked

Vendored code, a binary copied into the image, a package installed by hand: the lockfile does not name them.

Decide reachability

A finding means a vulnerable version is present, not that your code calls it. VEX records that judgment; it does not make it.

Replace the mapping

The PyPI identity is only as good as conda-forge's mapping. Where there is none, it says so.

Look at your own SBOM: is the hole there?

```
pixi global install pixi-sbom
```

```
uvx pixi-sbom --lockfile uv.lock
```

```
uses: millsks/pixi-sbom@v1
```

```
repo: https://github.com/millsks/pixi-sbom-pre-commit
```

Docs, the field manual and a hands-on lab: millsks.github.io/pixi-sbom

Source, examples and the paper: github.com/millsks/pixi-sbom